

Problem 1: Social Network Connection

1. Graph Representation

Use an **Adjacency List (Undirected Graph)**

Why:

- Sparse graph (users don't connect to everyone)
- Efficient neighbor lookup $\rightarrow O(\text{degree})$
- Saves space vs matrix $\rightarrow O(V + E)$

Representation:

Alice $\rightarrow$ Bob, Charlie

Bob $\rightarrow$ Alice, David

Charlie $\rightarrow$ Alice, Eve

David $\rightarrow$ Bob, Eve

Eve $\rightarrow$ Charlie, David

2. Find all friends of Alice

👉 Just return adjacency list of Alice

Algorithm:

```
friends(Alice):  
    return adj[Alice]
```

Output:

[Bob, Charlie]

3. Check if Bob and Eve are directly connected

👉 Check if Eve exists in Bob's adjacency list

Algorithm:

```
isConnected(Bob, Eve):  
    for friend in adj[Bob]:  
        if friend == Eve:  
            return true  
    return false
```

Output: False

4. Shortest path (degree of separation)

👉 Use **BFS (unweighted graph)**

Algorithm:

BFS(start, target):

queue $\leftarrow$ [(start, 0)]

visited $\leftarrow$ set()

while queue not empty:

node, dist $\leftarrow$ pop

if node == target:

return dist

for neighbor in adj[node]:

if not visited:

push(neighbor, dist+1)

return -1

Example: Alice $\rightarrow$ Eve

Alice $\rightarrow$ Charlie $\rightarrow$ Eve

Distance = 2

Problem 2: Course Prerequisite System

1. Graph Representation

👉 **Directed Graph (Adjacency List)**

CS101 $\rightarrow$ CS102, CS201

CS102 $\rightarrow$ CS202

MATH101 $\rightarrow$ CS201

2. Cycle Detection

👉 Use **DFS with recursion stack**

Algorithm:

DFS(node):

if node in recursionStack → cycle found

if node in visited → return false

mark visited + recursionStack

for neighbor:

if DFS(neighbor):

return true

remove from recursionStack

return false

3. Prerequisites for CS202

👉 Reverse traversal OR DFS from dependencies

Algorithm:

findPrereq(course):

for each parent:

recursively collect all parents

Output:

CS101 → CS102 → CS202

Answer: [CS101, CS102]

4. Topological Sort

👉 Use **Kahn's Algorithm (BFS)**

Steps:

1. Compute in-degree
2. Push nodes with 0 in-degree
3. Remove and update

Valid Order Example:

CS101, MATH101, CS102, CS201, CS202

Problem 3: City Road Network

1. Graph Type

👉 Directed + Weighted Graph

- One-way → directed edges
 - Two-way → add edges both directions
 - Distance → weight
-

2. Reachable nodes from A

👉 Use DFS or BFS

Algorithm:

```
DFS(A):  
  visit A  
  for neighbor:  
    DFS(neighbor)
```

Reachable:

A → B, C, D, E

3. Fewest turns (BFS)

👉 Treat each edge equally (ignore weights)

Path Example:

A → D → E

(2 steps, fewer turns)

4. Why DFS fails for shortest path

- DFS explores deeply, not optimally
- May find longer path first

Example:

A → B → C → E (longer)

while BFS finds:

A → D → E (shortest)

Problem 4: Island Counter

1. Graph Model

- Each cell = vertex
 - Edge = adjacent land (up/down/left/right)
-

2. DFS Approach

for each cell:

```
if grid[i][j] == 1:  
    DFS(i, j)  
    count++
```

DFS(i, j):

```
if out of bounds or water:  
    return
```

mark visited (0)

explore 4 directions

3. BFS Approach

BFS(i, j):

```
queue ← (i, j)  
mark visited
```

while queue:

```
    explore neighbors
```

4. Complexity

Approach	Time	Space
----------	------	-------

DFS	O(N×M)	O(N×M) recursion
-----	--------	------------------

BFS	O(N×M)	O(N×M) queue
-----	--------	--------------

5. Diagonal Connections

👉 Add 4 more directions

Total directions = 8

Effect:

- Fewer islands (more connected regions)

Problem 5: Network Packet Routing

1. Representations

Adjacency List

R1 → R2, R3

R2 → R1, R4

R3 → R1, R4

R4 → R2, R3, R5

R5 → R4, R6

R6 → R5

Adjacency Matrix

	R1	R2	R3	R4	R5	R6
R1	0	1	1	0	0	0
R2	1	0	0	1	0	0
R3	1	0	0	1	0	0
R4	0	1	1	0	1	0
R5	0	0	0	1	0	1
R6	0	0	0	0	1	0

2. Space Complexity

- Adjacency Matrix → $O(V^2)$
- Adjacency List → $O(V + E)$

3. Check if network is connected

👉 Use BFS/DFS from any node

if visited nodes == total nodes:
connected

4. Alternative path (R4–R5 fails)

👉 Remove edge and run BFS

Graph breaks into two parts → No path

R1 → R2 → R4 (component 1)

R5 → R6 (component 2)

So **no alternative path exists**

5. Minimum hops (BFS)

👉 Shortest path in unweighted graph

Path:

R1 → R2 → R4 → R5 → R6

Hops = 4